Swinburne University of Technology

School of Science, Computing and Emerging Technologies

MIDTERM COVER SHEET

Subject Code: COS30008
Subject Title: Data Structures and Patterns
Assignment number and title: Midterm: Solution Design & Iterators
Lecturer: Dr. Markus Lumpe

Your name:_____ **Your student ID:**_____

Marker's comments:

Problem	Time Estimate	Marks	Obtained
Reading	10 minutes		
1	40 minutes	62	
2	60 minutes	119	
Total	110 minutes	181	

Instructions:

Read the midterm test brief carefully. There are two problems.

You do not need to test or compile your solution. You can use a text editor like Notepad++ to write your solution. You can also use Visual Studio or Xcode to write your solution. Using these environments may be beneficial, as they come with syntax highlighting and code completion.

We do not accept AI-generated code. You must submit original code.

You can test your code with the provided test drivers in `Main.cpp`. The build setting must be C++20. Use caution, as testing your code can be time-consuming and distracting in the midterm test. Rely on your intuition and knowledge. Do not let the compiler destroy your confidence.

Midterm: Solution Design & Iterators

Julius Caesar (100BC – 44BC) used a substitution cipher in his private correspondence, in which each letter in the plain text was replaced by a letter some fixed number of positions down the alphabet.

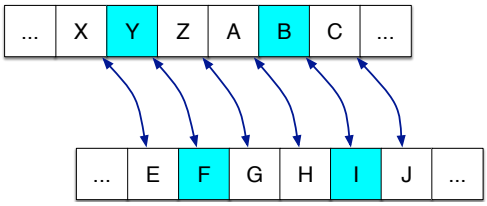


Figure 1: Caesar Cipher - Letter plus 7 Positions.

Figure 1 illustrates the Caesar cipher. With an offset of 7, the Caesar cipher maps a 'B' to an 'I' and a 'Y' to an 'F', and vice versa. The Caesar cipher is easily reversible by inverting the offset.

Notice the circular nature of the encoding. We can view a 26-letter alphabet as a fixed-size buffer with 26 letters. Encoding the letter 'B' with an offset of 7 creates a value that corresponds to the letter 'I' or the 9th letter in the alphabet.

On the other hand, encoding the letter 'Y' with an offset of 7 produces a value that maps to the 32nd character, which is clearly out of range. We must map this value back into the alphabet. We can do this by subtracting 26 from the value. This correction maps the value of 'Y' + 7 to 'F', the 6th character in the alphabet.

A small extension can turn the Caesar cipher into an *autokey cipher*. Rather than using a fixed offset all the time, we can use a keyword and the message for the cipher process. The letters of the upper-case alphabet, 'A' … 'Z', can be turned into offsets in the range 0 … 25. Subtract the letter 'A' from the upper-case alphabet, 'A' – 'A' … 'Z' - 'A', and the result is a set of offsets in the range 0 … 25.

With this mapping of upper-case letters to offsets, we can define an autokey Caesar cipher. We start with the keyword. We allow only letters for the keyword. For example, a valid keyword is the sequence of characters 'Berlin'. First, all letters have to be converted to upper-case characters: 'BERLIN'. Next, we subtract the letter 'A' to obtain the sequence of offsets 1, 4, 17, 11, 8, 13, the initial keyword for the autokey Caesar cipher.

Next, we want to encode the message: "Julius Caesar was a Roman general."

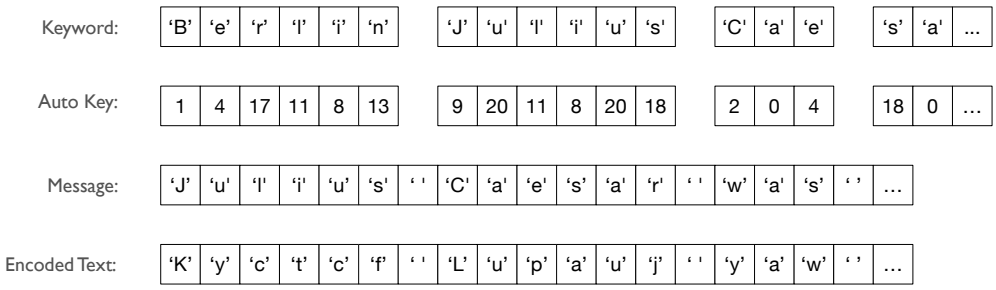


Figure 2: Autokey Caesar Cipher - Keyword 'Berlin'.

A message may use all characters of the printable ASCII alphabet, including spaces, numbers, and punctuation symbols. Note, however, that the Caesar cipher only affects letters. We proceed by inspecting every character in the message. If it is a non-letter character, we return it as is. No keyword offset is consumed. If we encounter a letter, we temporarily convert it to

upper-case. Using the corresponding keyword offset, we calculate the encoded character. For example, 'J' + 1 produces 'K'. The offset 1 is removed from the keyword. If the letter we started with is lowercase, we convert the encoded character to lowercase. In the final step, we append the keyword with the value of 'J' – 'A', the offset 9. This is the autokey feature of the cipher: the current character becomes an additional element of the keyword. The autokey behaves like a moving window.

Figure 2 illustrates the process. Only letters are encoded. The space character (and any punctuation symbol) does not get encoded or consume keyword offsets. We also see that the same character can get encoded differently when the offsets do not match. The letter 'u' in 'Julius' is first encoded as 'y' and then as 'c'. This is a result of the different offsets, 4 and 8. It is also possible to map a character to itself if the offset is 0.

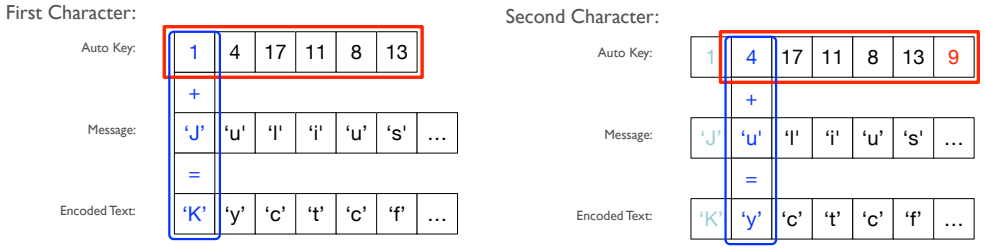


Figure 3: Autokey Moving Window.

When the autokey Caesar cipher finishes, the autokey will contain the remaining offsets. In general, the number of offsets corresponds to the length of the keyword. In the example, the keyword is 'Berlin'. It has a length of 6. Hence, the autokey ends with 6 offsets: 4, 13, 4, 17, 0, 11, which correspond to the sequence 'eneral', the last six characters in the message encoded by the autokey Caesar cipher.

The autokey Caesar cipher can be implemented using C++ iterators. We use an input iterator for the autokey and a forward iterator for the cipher process. The autokey iterator behaves like a pointer that is positioned at the current offset. We can access the current offset by dereferencing the autokey iterator. We position the autokey iterator on the next offset by incrementing it. This is the behavior of a standard input iterator.

The autokey iterator maintains a fixed-size array internally, defined as a circular buffer. Initially, this array is filled with the keyword offsets. The autokey iterator maintains two index variables to access the current offset and to indicate the next available slot in the circular buffer. The design of the autokey iterator guarantees that the index variables can never refer to the same element in the circular buffer. Remember, a quintessential aspect of a circular buffer is that if an index goes out of bounds, it is "warped back" into the array by resetting it to 0. As we process a message, the circular buffer is periodically updated with message offsets, and the indices for the current offset and the next free slot change. The autokey iterator never stops, unless we stop processing a message.

For the Caesar cipher process, we use a standard forward iterator. This iterator receives a character keyword array and a pointer to a message string. The character keyword array serves as the initializer for the autokey, and the pointer to a message string represents the underlying collection. An index variable captures the position of the iterator. The dereference and prefix increment operations are more complicated this time. The dereference operation performs the Caesar encoding of the current message character. The process follows the blue box in Figure 3. The prefix increment first updates the autokey if the current message character is a letter and advances the autokey to the next position. Then the forward iterator moves to the next message element. These two refinements allow us to systematically traverse the underlying collection (the message string) and apply the Caesar cipher on it.

Problem 1 (62 marks)

Start with the template class `KeyIterator`. It defines an input iterator to construct the autokey feature for the autokey Caesar cipher.

The template class `KeyIterator` is parameterized over the size of the underlying array. Using this approach frees us from passing the size of the keyword array explicitly to the iterator constructor. In addition, all range checks can refer to the template parameter `N`. Please note that the size of the array `fKey` is `N+1`. We use the array `fKey` as a circular buffer. By increasing the size of this buffer by one, the two indices `fIndex` and `fUpdateIndex` can never refer to the same element. There is always a gap of at least one element. (The length of the shortest valid keyword.)

A suggested specification for the template class `KeyIterator` is shown below:

```
#pragma once

#include <cctype>

template<size_t N>
class KeyIterator
{
public:

  using difference_type = std::ptrdiff_t;
  using value_type = size_t;
  using iterator = KeyIterator;

  KeyIterator( const char aInit[N] ) noexcept;          // 18 marks

  size_t operator*() const noexcept;                    // 4 marks

  iterator& operator++() noexcept;                      // 12 marks
  iterator operator++(int) noexcept;                    // 10 marks

  iterator& operator+=( char aKey ) noexcept;           // 18 marks

private:

  value_type fKeys[N+1];
  size_t fIndex;
  size_t fUpdateIndex;
};
```

This is an input iterator.

The constructor receives a character array of size `N`. You use the parameter `aInit` to initialize the array `fKeys`. You can safely assume that the array only contains letters. You can check it using `std::isalpha(c)`, defined in the header '`cctype`', but it is not required here. However, you need to explicitly convert all characters in the array `aInit` to upper case using `std::toupper(c)`, also defined in the header '`cctype`'. You do this while copying the elements. Remember, the array `aInit` stores letters, whereas the array `fKeys` stores offsets. To obtain offsets, you subtract the character `'A'` from the keyword characters. After initialization, the iterator must be positioned on the first offset, and the update index must point to the next available slot in the array `fKeys`.

The deference operator and the postfix increment operator are defined in the standard way.

The prefix increment operator naturally advances the iterator position. Remember, however, that the array `fKeys` defines a circular buffer. If the iterator index goes out of bounds (the size of the array `fKeys` is `N+1`), the index must "warp back" to index 0.

The operator **+=**, also called auto-increment, allows us to model the autokey feature with this iterator. The operator **+=** receives a character as the argument. You can safely assume that the character `aKey` is a letter. (It is the protocol defined in this midterm test.) However, you always have to convert it to upper case and subtract the character 'A' to obtain an offset before storing it at the next update index. Once the offset has been stored, you advance the update index. Again, the update index can go out of bounds. In this case, it must be "warped back" to index 0. Finally, a reference to the updated iterator object is returned.

You do not need to check your implementation in the midterm. However, if you want to, the file `Main.cpp` contains a test function to check your implementation. Uncomment **`#define`** `P1` and compile your solution. Your program should produce the following output:

```
Test AutoKey:


Initial lAutoKey: 1 4 17 11 8 13
lAutoKey after message: 9 20 11 8 20 18 2 0 4 18 0 17 22 0 18 0 17 14 12 0 13 6 4 13 4 17 0 11


Test AutoKey complete.
1 Test(s) completed.
```

The output confirms that the keyword produces the offsets 1, 4, 17, 11, 8, and 13. The second sequence corresponds to the characters in the message. Only letters are sent to the autokey.

The test function `runP1()` contains elements that may help you in the definition of the template class `KeyIterator`.

Problem 2 (119 marks)

Assume your definition of template class `KeyIterator` meets the specification. You can independently implement the solution for this problem.

We wish to define a forward iterator that performs an autokey Caesar cipher along the systematic traversal of a message string.

We define this iterator in class `AutoCaesarIterator`. A suggested C++20-compliant specification of the class `AutoCaesarIterator` is shown below:

```
#pragma once

#include <string>

#include "KeyIterator.h"

class AutoCaesarIterator
{
public:

  using difference_type = std::ptrdiff_t;
  using value_type = char;
  using iterator = AutoCaesarIterator;

  static constexpr size_t KEY_LENGTH = 6;

  AutoCaesarIterator( const char aInit[KEY_LENGTH],
                      const std::string* aCollection = nullptr ) noexcept;
                                                                  // 12 marks
  value_type operator*() const noexcept;                          // 36 marks

  iterator& operator++() noexcept;                                // 18 marks
  iterator operator++(int) noexcept;                              // 10 marks

  bool operator==( const iterator& aOther ) const noexcept; // 15 marks

  iterator begin() const noexcept;                                // 14 marks
  iterator end() const noexcept;                                  // 14 marks

private:

  const std::string* fCollection;
  const char* fInit;
  KeyIterator<KEY_LENGTH> fKeys;
  size_t fIndex;
};
```

We implement the class `AutoCaesarIterator` in a C++ file. Modern development environments suggest '`AutoCaesarIterator.cpp`' as the file name.

The iterator defines four member variables. The member variable `fCollection` is a pointer to the underlying collection. Here, it is a message string. We also store a pointer to the first element of the keyword array in `fInit`. We need access to it in the `begin()` method. The member variable `fKeys` denotes the autokey for the autokey Caesar cipher. The member variable `fIndex` is the position of the forward iterator. It must be set to the first item in the underlying collection (even if `fCollection` is **`nullptr`**).

The dereference operator returns the element that the iterator is currently positioned on. Here, it must also perform the Caesar cipher. Hence, if the character to be returned is a letter

(you can test it with `std::isalpha(c)`), it must be encoded first using the Caesar cipher. The following pseudo illustrates the process:

set `Result` to the current character

if `Result` is a letter

then

set `lIsLower` to **`true`** if `Result` is lowercase

make `Result` an upper-case letter

add the current offset from the autokey to the value of `Result`

if `Result` is greater than the letter `'Z'`

then

subtract 26 from `Result`

if `lIsLower` is true

then

make `Result` a lowercase letter

return `Result`

Please note that the deference operator does not advance or change the autokey. Repeated calls to the dereference operator without advancing the iterator intermittently have to return the same value. (This is a feature that we do not exploit when using iterators, but it is consistent with pointer semantics.)

The prefix increment operator advances the iterator. Unlike in previous examples, it has to perform two operations. If the iterator is currently positioned on a letter in the underlying collection, we must add it to the autokey `fKeys` using its auto-increment operator. A message character becomes part of the keyword. Next, we advance the autokey `fKeys` to the next position. Advancing the iterator requires consuming the current keyword offset. Finally, we increment the iterator position and return a reference to the updated iterator object.

The postfix increment operator is defined in terms of the prefix increment operator, but must return the old iterator.

The equivalence operator returns true if the left and right iterators are positioned on the same element within the same collection, and both use the same keyword. We do not need to check the autokey. The autokey is always in sync.

The `begin()` and `end()` methods return a copy of the current iterator positioned at the first element and set to the end of the iteration, respectively. The `begin`-iterator also needs to reset the autokey. You will need to use the following expression

```
KeyIterator<KEY_LENGTH>(fInit)
```

to reset the autokey in the `begin`-iterator. This expression denotes a constructor call for the template class `KeyIterator` specialized to size `KEY_LENGTH`, a constant value defined in class `AutoCaesarIterator`. Even though we program in C++20, we need to specify the template parameter for the class template `KeyIterator`. The compiler cannot deduce it. It is `KEY_LENGTH` or 6 in the midterm.

The implementation of the `end()` method is identical to the approach we used in Tutorial 6.

You do not need to check your implementation in the midterm. However, if you want to, the file `Main.cpp` contains a test function to check your implementation. Uncomment **`#define`** `P2` and compile your solution. Your program should produce the following output:

```
Test AutoCaesarIterator:

Keys:     1  4 17 11  8 13  9 20 11  8 20 18  2  0  4 18  0 17 22  0 18  0 17 14 12  0 13  6
Letters:  J  u  l  i  u  s  C  a  e  s  a  r  w  a  s  a  R  o  m  a  n  g  e  n  e  r  a  l

Clear:   Julius Caesar was a Roman general.
Encoded: Kyctcf Lupauj yaw s Rfiaf gvbqrnr.

Test AutoCaesarIterator complete.
1 Test(s) completed.
```

The output demonstrates the effect of the autokey Caesar cipher. The first sequence aligns autokey offsets with message characters.

The second sequence prints the result of encoding the sentence 'Julius Caesar was a Roman general.' using the keyword 'Berlin'.

The test function `runP2()` contains elements that may help you in the definition of the class `AutoCaesarIterator`.

Submission requirements

The midterm test is timed. The normal time allowance is 110 minutes. Canvas submissions will be allowed until two minutes before the end of your session.

Follow the instructions on Canvas. Submit your work before the submission window closes.

You can use text files, header or source files, and even PDF files.

Your solution will not be tested against solution artifacts. It will be assessed like any written exam. Minor syntax errors will not affect your mark. The marking process will test how close your code is to a prototype solution.